

Chapter 4: Foundations of Computer Science Algorithms

1. Introduction to Searching Algorithms

Searching is one of the most fundamental operations in computer science. Given a collection of elements, the goal of a searching algorithm is to retrieve the location of a target element, or determine that it does not exist. The simplest searching method is Linear Search, which checks each element sequentially. However, when dealing with sorted collections, a far more efficient method called Binary Search can be utilized.

2. Binary Search Concept & Execution

Binary Search is a divide-and-conquer algorithm that finds the position of a target value within a sorted array. It compares the target value to the middle element of the array. If they are unequal, the half in which the target cannot lie is eliminated, and the search continues on the remaining half, repeating this process until the target value is found. If the search ends with the remaining half being empty, the target is not in the array. Because Binary Search cuts the search space in half with each step, its time complexity is $O(\log n)$, where n is the number of elements in the array. This is significantly faster than Linear Search, which has a time complexity of $O(n)$.

3. Recursion: Theory & Practice

Recursion is a programming technique where a function calls itself, directly or indirectly, to solve a problem. A recursive function must have two main components to run successfully: (1) The Base Case, which terminates the recursion and prevents infinite loops. (2) The Recursive Case, which reduces the problem size and calls the function again. Without a proper basecase, a recursive function will exhaust the stack memory, resulting in a stack overflow error.

A classic example of recursion is the Fibonacci sequence, where each number is the sum of the two preceding ones: $F(n) = F(n-1) + F(n-2)$, with base cases $F(0) = 0$ and $F(1) = 1$. Recursion is highly useful for traversing hierarchical structures like trees and graphs, but can incur overhead due to stack allocation for each recursive step.

4. Time Complexity & Big O Notation

Time complexity is a computational metric that describes the amount of computer time it takes to run an algorithm. It is commonly expressed using Big O Notation, which characterizes functions according to their growth rates. Common complexity classes include: $O(1)$ for constant time, where execution time remains independent of input size; $O(\log n)$ for logarithmic time, typical of binary search; $O(n)$ for linear time, typical of linear search; and $O(n \log n)$ for linearithmic time, typical of efficient sorting algorithms like Merge Sort and Quick Sort.